WHITE PAPER · JUDGES EDITION

A private, on-chain reservation primitive

One trusted source of truth for *who reserved what* — fed to many mutually-distrusting sales channels, without putting prices, terms, or personal data on-chain.

✓ VETTED NETWORK

Status
Live on NEAR testnet · calculating-hand.testnet

Hosted demo
vrx.poolo.org (seeded sandbox)

Judge demo
npm run demo — one command

As of
November 2026

FOR THE JUDGES, IN ONE LINE

A public ledger of reservations that leaks none of the deal. The chain proves *that* a reservation exists and *who* holds it; the meaning stays confidential off-chain. It is built and running — testnet contract live, hosted demo reachable, one-command walk-through.

ABSTRACT

In one page

VRX is a reservation primitive: a callable NEAR smart contract that records *who reserved what*, paired with an off-chain portal that carries everything the chain must not — identities, prices, specifications and deal terms.

An operator with **unique, non-substitutable inventory** (unit 312 is not unit 311) needs to feed live availability to many sales teams and agencies who do not trust one another, and let them browse, book, reserve, take a deposit and hold — without double-booking, without leaking the commercial or personal details of a deal, and without a channel being able to go around the operator. Generic booking software centralizes trust in one vendor's database; a naïve on-chain listing leaks the very terms that make a deal confidential. VRX reconciles both.

The design answer is a strict **split**. The chain holds only opaque hashes and neutral account ids: it proves *that* a reservation exists and *who* holds it, and nothing else. The meaning — the *what*, the *how-much*, the *with-whom* — lives off-chain under server-side authorization and field-level masking. Privacy is therefore not a promise bolted on; it is a property of what is and is not written to the ledger.

> **THE SHAPE OF THE CLAIM**
>
> Real estate (off-plan property) is the beachhead and the working proof. But the contract never "knows" it is real estate — every real-world reference is an opaque hash — so the same primitive serves cars, livestock, equipment, event slots: any Class-A, non-substitutable inventory.

ON-CHAIN	Reservation / hold / exclusive / attestation registry — opaque hashes only.
OFF-CHAIN	The portal: role-scoped apps, sealed records, masking, consent ceremonies.
CONTRACT	Rust / near-sdk 5.26.1; state schema V1 → V7 via borsh-safe migrations.
LIVE	`calculating-hand.testnet` · hosted sandbox at `vrx.poolo.org`.
POSTURE	VRX facilitates and seals; it does not adjudicate disputes or warrant enforceability.

Status language throughout is honest: **built** means it runs today; **in progress / planned** means it is designed or partially built but not yet usable as described. The hosted demo is a seeded *sandbox* — write actions are simulated; the contract itself runs for real on NEAR testnet.

WHY IT WINS

Three tracks, one build

VRX is not three demos wearing a trench coat. It is a single system whose core property — proof on-chain, meaning off-chain — lands squarely in three judging tracks at once.

Open Innovation

VRX is a genuinely new on-chain primitive, not a fork of an existing pattern: a reservation registry for **non-substitutable** inventory, shared across mutually-distrusting channels. The contract is domain-agnostic by construction (every real-world reference is an opaque hash), so the same deployment serves off-plan property today and cars, livestock, equipment or event slots unchanged. The contract is real — Rust, near-sdk 5.26.1, seven borsh-safe state migrations, live on testnet — not a slide.

Privacy & Private AI

Privacy is the moat, not a feature. **No personal data, no prices, no deal terms ever go on-chain** — only opaque hashes and neutral account ids. The confidentiality that makes a deal a deal is preserved *while* the shared truth is public. Sensitive imagery is served through a two-stage reveal and is provenance-stamped so a leaked image is traceable — and the stamping mechanism is deliberately private (the moat). An outsider's own AI can evaluate fit off-platform from a published machine-readable guide, without being handed private data.

Real-World Money

VRX moves real commerce: live availability, deposits taken against a timed hold, commission agreements anchored on-chain (percentages and splits stay off-chain), a two-sided payment-attestation ledger, and an in-system $VRX credit (claim / send / spend). Booking capacity is itself a scarce, tamper-evident token that cascades down the trust chain. On-rail settlement and the on-chain NEP-141 form of $VRX are **planned** — stated honestly, not implied as shipped.

> **THE THROUGH-LINE**
>
> Each track is satisfied by the *same* architectural choice — the on-chain / off-chain split — so the privacy claim, the innovation claim and the money claim all rest on one proven mechanism rather than three separate bolt-ons.

THE PROBLEM

Why nothing already does this

Selling scarce, one-of-a-kind inventory through many channels that do not trust each other breaks the tools everyone already uses — in three ways that compound.

- ✓ **Double-booking a unique thing.** When each item is unique — a specific apartment, a specific pedigree animal, a specific machine — two channels reserving "312" at once is a hard conflict. The market needs one authoritative answer to "is this exact unit still available?" that every channel reads the same way, in real time.
- ✓ **Privacy leakage.** A shared spreadsheet or single vendor database exposes who is buying what, at what price, through whom. Putting the registry on a public chain naïvely is worse — prices, parties and terms become world-readable forever.
- ✓ **Circumvention.** A channel that learns the end-buyer and the operator can be tempted to cut the introducer out and deal direct. Hiding the buyer until a deal is firm is itself a privacy problem that must be reconciled with the operator's need to confirm a booking.

Three existing approaches, and where each leaks

APPROACH	GIVES YOU	WHAT IT COSTS
Generic booking / CRM software	A shared calendar and pipeline.	Centralizes trust in one vendor's database; the vendor (and often rival channels) see who buys what at what price. No independently-provable record.
Naïve on-chain listing	A tamper-evident, independently-verifiable record.	Publishes prices, parties and terms to the world forever. Confidentiality is destroyed to gain the shared truth.
Point-to-point / off-platform deals	Privacy by obscurity.	No shared availability, so double-booking and circumvention are endemic; no neutral record when a dispute arises.

VRX'S POSITION

VRX takes the tamper-evident record from the on-chain approach and the confidentiality from the off-platform approach and reconciles them with a **split**: the chain carries proof-of-state over opaque keys; a private, access-controlled off-chain layer carries the meaning. The moat is the discipline of *what is never written to the ledger at all*, plus a vetted operator network layered on top.

ARCHITECTURE · THE ONE FIGURE THAT MATTERS

The on-chain / off-chain split

One rule governs the whole system: **meaning never touches the chain**. The figure shows what lives where, and the one-way binding between them.

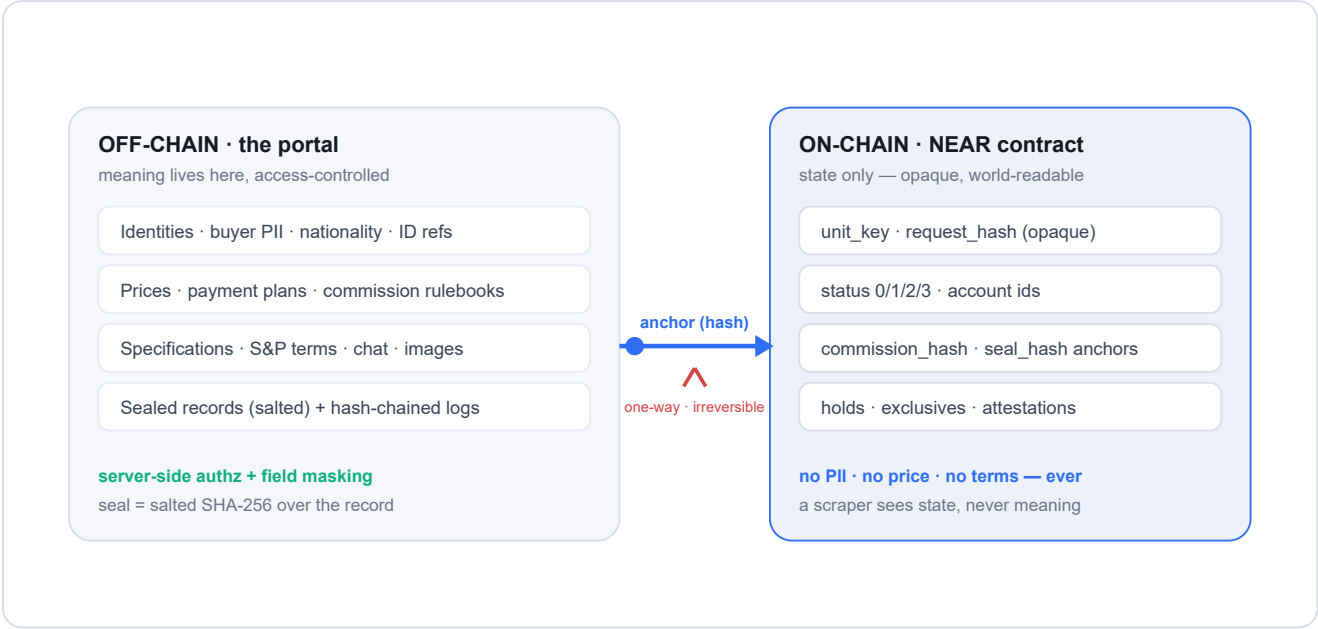


Figure 1 — The split. Off-chain records are sealed; only their hashes are anchored on-chain. The binding is one-way.

WHY ONE-WAY MATTERS

An anchored `seal_hash` lets anyone later prove a specific off-chain record is the one that was committed (re-seal it, compare). It never lets anyone recover the record from the hash. Tamper-evidence without disclosure is the whole trick.

ARCHITECTURE

One booking, end to end

Follow one booking from live availability to a confirmed hold, watching where meaning stays and only a hash moves.

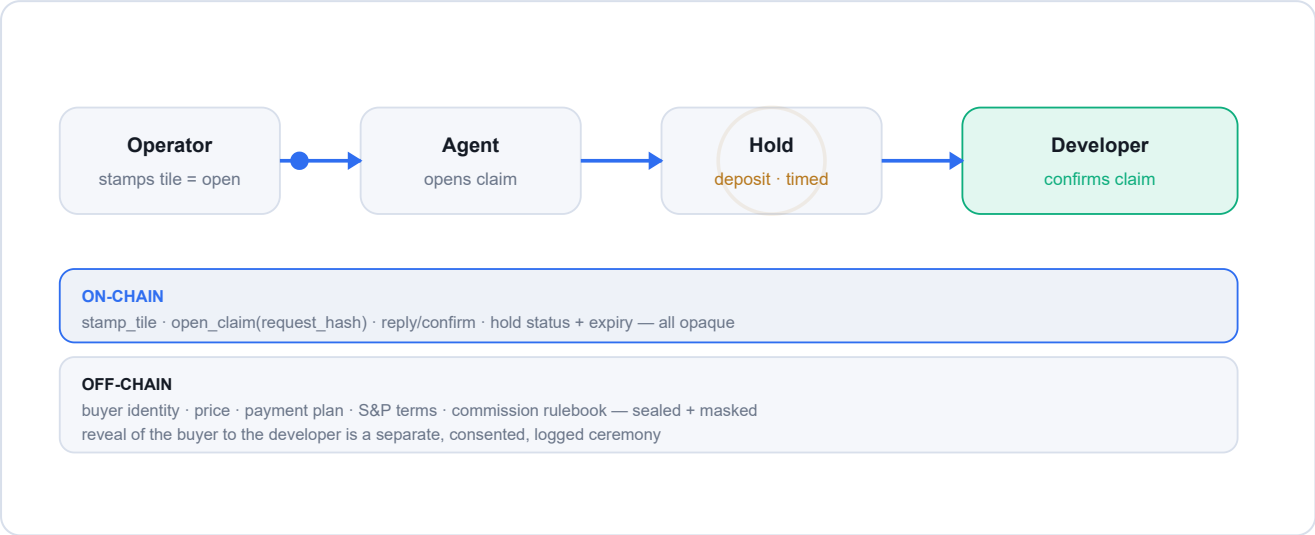


Figure 2 — One booking, end to end. Only opaque keys + state cross into the on-chain lane.

The step sequence

1. Operator `stamp_tile` → unit `open` (opaque `unit_key`).
2. Agent `open_claim` (or `burn_pass_hold` on the booking-token path) → an opaque `request_hash` anchors the off-chain request; the unit moves to `hold` with a timed expiry.
3. Off-chain: deposit recorded, buyer profile sealed, masked to the developer by default.
4. Developer `reply_claim` / `confirm_claim` → state advances; a consented reveal of the buyer is a separate ceremony.
5. On completion `finalize_tile` → `done`; expired holds auto-release back to `open`.

PRIVACY

The privacy model

Two distinct protections, applied per surface: on-chain opacity by construction, and off-chain access control by the server.

Protection 1 — on-chain opacity (by construction)

Everything on the contract is an opaque SHA-256 hash (`unit_key`, `request_hash`, `seal_hash`, `commission_hash`) plus neutral account ids and status codes. A chain scraper sees "*account X holds an exclusive on hash 07f2…*" — state, never meaning. The building, unit, price, party and terms cannot be derived from the chain. This moat holds by design.

Protection 2 — off-chain access control (by the server)

The plaintext that carries meaning lives off-chain, protected by **server-side authorization + field-level masking**. The engineering question for every counterparty-facing surface is: *is this read scoped to a caller entitled to it, and masked server-side before the wire?* A UI blur is theatre; masking happens on the server, so the plaintext never reaches the browser for a masked field.

Supporting mechanisms

- ✓ **Salted seals** — tamper-evidence without reversibility; a per-record salt plus an optional global pepper held separately, so a breach that leaks the store alone cannot re-derive seals.
- ✓ **Encryption-at-rest** for identity-document images — per-file key under an offline-sealed master key.
- ✓ **Append-only, hash-chained chat** — the dispute trail is tamper-evident; editing or reordering a past message is detectable.
- ✓ **Provenance-stamped imagery** — a leaked sensitive image is traceable back to the recipient it was served to; the stamping *mechanism* is deliberately private (the moat), and it is applied under a two-stage reveal (public exterior always; sensitive imagery only once a buyer is offer-eligible and consented).

PRIVACY

What an outsider can and cannot see

OBSERVER	CAN SEE	CANNOT SEE
Chain scraper	That a reservation/hold/exclusive exists; which account holds it; status + version.	Unit, building, price, party, terms — anything meaningful.
A rival channel	Shared availability (open/held).	Another channel's buyer, price or terms.
Authority-review role	The *vetted state* for oversight.	Private buyer data — the moat extends to the regulator view.

THE HONEST LIMIT

On-chain opacity is absolute by construction. The off-chain layer's confidentiality rests on server-side authorization, not cryptography — so VRX must be trusted to run it correctly, and a standing security/privacy audit exists to make that trust checkable. That is where almost all the engineering attention goes.

THE NETWORK

Roles & the vetted network

VRX is not only a contract — it is the vetting entity and the network operator. Identities enter the system only through VRX; the chain then anchors the fact of a sign-off, never its contents.

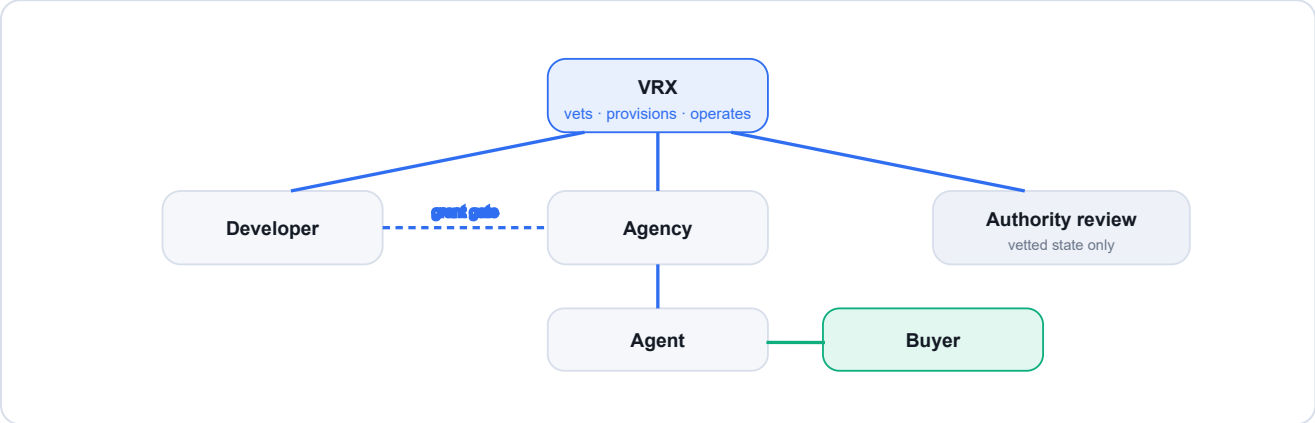


Figure 3 — Roles & trust network. VRX vets the whole system; developers gate agencies; agencies supervise agents; agents serve buyers.

Eight participant roles: **Developer** (owns inventory, grants access, confirms bookings, may sell direct), **Direct-seller** (the developer's own desk), **Agent** and **Agency**, **Buyer** (a state, not a vetted role — any wallet), **Direct-buyer** (a walk-in meeting a light anti-abuse bar **planned**), **CEO / Approver** (an approval trust-dial), and **Authority Review** (sees vetted state for oversight, never private data). Onboarding a node or org is owner-gated; KYC is a human-led off-chain precondition, and nothing identifying ever lands on-chain.

Non-circumvention & booking tokens

Trust is enforced by **visibility, not disclosure**: the developer and agency see *who* holds a booking, not necessarily the private buyer, until a consented sealed reveal. Reservation capacity is itself a scarce, tamper-evident **booking token** that cascades down the trust chain (developer → agency → agent) and returns up on release — shown as "coins" in the wallet, backed on-chain by a non-transferable pass with burn + rebirth.

STATUS

What is built, honestly

Built means it runs today; the rest is labelled truthfully. No **planned** item is dressed up as shipped.

CAPABILITY	STATUS
On-chain reservation / hold / exclusive / attestation registry, live on NEAR testnet	**built**
Role-scoped dashboards for all participant types; browse → book → reserve → hold funnel	**built**
Attested off-chain records; commission + settlement *projection*	**built**
Booking tokens (scarce, cascading, tamper-evident) shown as "coins"	**built**
S&P contract authoring (payment plans, packages, concessions, termination terms)	**built**
Vetting / listing gate; change-notice + oversight; off-chain enforcement	**built**
$VRX off-chain credit ledger (claim / send / spend), fixed 200M treasury-held	**built**
Provenance-stamped imagery with a two-stage reveal	**built**
Participation / node-license funnel + a live ecosystem map of the network	**built**
Richer agency console (per-agent cards, chat, scheduling/dispatch)	**in progress**
Real-wallet verified sign-in (NEP-413) — seam built, default off	**in progress**
Real on-chain settlement rail; secondary market; multi-jurisdiction packs	**planned**
$VRX on-chain NEP-141 fungible-token form	**planned**
Determined-attacker-resistant watermark + read-side image analyzer	**planned**

CONTRACT MATURITY

The contract (Rust, near-sdk 5.26.1, `#[near]` macros) has grown through seven state schema versions, each added by an owner-only, borsh-safe migration that never touches an existing record. State changes emit structured NEP-297 events carrying only data already on-chain — opaque ids, account ids, status, version — never PII, amounts or terms.

Path: testnet (`calculating-hand.testnet`) → mainnet. No known privacy/correctness leak is open; the deferred items are deliberate and sequenced behind a dependency (for example, the authentication seam is built and default-off because the demo is deliberately wallet-less, with authorization already enforced server-side).

FOR THE JUDGES

Run it · legal posture · contact

The one-command judge demo

NPM RUN DEMO

From the portal, `npm run demo` runs a reproducible end-to-end walk of the privacy funnel (availability → book → hold → confirm) in a near-workspaces sandbox — one command, no setup clicks. The hosted demo is at `vrx.poolo.org`: enter wallet-free via `?enter=1`, assume any role (Developer / Agency / Agent / Buyer) from the top bar, and walk one deal end to end. **Write actions in the hosted sandbox are simulated**; the contract on NEAR testnet is the honesty anchor, not the sandbox clicks.

Legal posture (structural)

- ✓ **A facilitator and a sealer.** VRX provides the private shared truth and the tamper-evident record. It records that a reservation, agreement or change happened, and seals it.
- ✓ **Not an adjudicator.** VRX does not decide disputes. It records the evidence (before/after, who was notified, the sealed declarations) — never the dispute's resolution.
- ✓ **Not a warrantor of enforceability.** VRX does not warrant that any underlying contract is legally enforceable. Parties' own terms and local law govern.
- ✓ **Privacy-by-design.** The architecture itself (opaque on-chain, masked off-chain) is the primary privacy control, not a policy promise layered on afterwards.

The one-paragraph pitch to a cold reader

VRX is the private source of truth for who reserved what. It turns a public blockchain's strength — a tamper-evident, independently-verifiable record — into something usable for confidential commerce by putting *only* opaque proof on-chain and keeping every meaningful detail in a vetted, access-controlled off-chain layer. Real estate proves it; the primitive is domain-agnostic. 312 is not 311.

AN AI FIT-CHECK, OFF-PLATFORM

A judge can hand their own AI the machine-readable guide served at `vrx.poolo.org/for-ai-agent.md` to evaluate whether VRX fits a use-case without driving the live app — it carries honest built / in-progress / planned status for every capability.

Contact: `vrx@vettedrealty.com` — the business behind the primitive is Vetted Realty (`vettedrealty.com`).